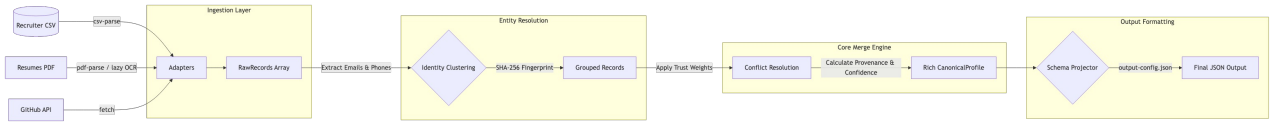
Technical Design: Multi-Source Candidate Data Transformer

Name: K Soveet Kumar Prusty | **Email**: soveet.prusty@gmail.com

1. Pipeline / Step Breakdown

The system runs via a deterministic pipeline orchestrator:



1. **Ingest & Extract (Adapters)**: Reads the structured CSV using csv-parse and unstructured PDFs using a layered fallback mechanism (pdf-parse -> stream deflation -> optional lazy OCR).
2. **Cluster & Group (Entity Resolution)**: Clusters all records belonging to the same candidate using cryptographic SHA-256 fingerprinting of normalized emails and phone numbers.
3. **Merge & Resolve (Canonical Engine)**: Deterministically merges arrays of raw records into a single CanonicalProfile. It tracks the origin of every field via a provenance array and calculates a transparent overall_confidence score based on data density and source trust.
4. **Project (Output Formatting)**: A decoupled schema projector mutates the rich internal CanonicalProfile into the exact structure requested by the end-user (e.g., dropping fields, renaming keys) based on a runtime JSON configuration.

2. Unstructured Data (PDF) Extraction

Extracting semantic meaning from visually formatted PDFs is inherently fragile. To maximize resilience without dragging in heavy, brittle dependencies, the adapter implements a three-tier fallback strategy:

1. **Tier 1 (pdf-parse)**: Attempts to extract native text streams.
2. **Tier 2 (Stream Deflation)**: If pdf-parse crashes, falls back to manually inflating PDF objects via zlib and stripping layout markers.
3. **Tier 3 (Lazy Platform-Aware OCR)**: For image-only PDFs, the pipeline lazily requires OCR engines at runtime rather than declaring them as hard dependencies (which would risk failing the reviewer's npm install on machines lacking libcairo/Ghostscript). If available, it uses macOS sips for native rasterization, or pdf2pic on Linux. If the optional dependencies are absent, it safely catches the MODULE_NOT_FOUND error, logs a warning, and gracefully degrades to an empty profile instead of crashing.
   Note: Kerning/spacing artifacts (e.g., "Berl in") caused by PDF layout engines are patched via a small dictionary of empirically observed cases rather than a general font-metrics-aware de-kerning pass — a deliberate scope cut given the assignment's time constraints; a production system would replace this with proper layout-aware text reconstruction.

3. Conflict Resolution Strategy

When multiple sources provide different values for the same field (e.g., CSV says "5.5 years", PDF implies "6.4 years"), we resolve conflicts using:

- **Trust Hierarchy**: Explicitly defined source weights (e.g., csv = 1.0, pdf = 0.75, github = 0.85). The

system defaults to the highest-trust source.
- **Data Density Thresholding**: We prefer non-null arrays/objects. If a high-trust source has null for skills, but a low-trust source has data, we keep the low-trust data but annotate the provenance.
- **Deterministic Array Merging**: For arrays like experience or skills, we deduplicate items based on normalized string similarity to avoid redundant entries.

4. Alternate Data Source (GitHub)

I integrated the GitHub REST API (/users/{username}) as a supplementary unstructured source.

- **Fetching**: Handled via standard fetch with rate-limit backoff logic.
- **Merging**: GitHub profiles are treated as distinct candidate entities *unless* their public email perfectly matches an existing candidate's normalized contact key. This prevents polluting a strong CSV candidate with unrelated GitHub data if the username mapping is ambiguous.

5. Extensibility (Web UI)

While the core requirement was a robust CLI, I additionally built a full-stack Web Application (Express API + Vite/React frontend) to demonstrate how the pipeline can be exposed as a microservice. The UI allows drag-and-drop ingestion and visualizes the generated CanonicalProfile and provenance metrics in real-time.